

# BÀI 2

# Golang & Kafka

## Nâng Cao

Consumer Group · Goroutine · Retry Pattern · Serialization

**Kế tiếp bài:** Nhập môn Golang & Apache Kafka  
**Phiên bản:** 2.0.0  
**Ngày:** Tháng 4, 2025  
**Yêu cầu:** [Go 1.22+](#) | [kafka-go](#) | [Docker](#)

# Mục Lục

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Mục Lục.....                                                    | 2  |
| Ôn Tập Nhanh & Folder Structure Mở Rộng.....                    | 4  |
| Folder structure mở rộng (phần mới in đậm).....                 | 4  |
| Chương 1: Consumer Group & Offset Management .....              | 5  |
| 1.1 Consumer Group là gì? .....                                 | 5  |
| 1.2 Rebalance — Kafka tự cân bằng khi có biến động .....        | 5  |
| 1.3 Commit Offset: Tự động vs Thủ công.....                     | 6  |
| 1.3.1 Code: Manual Commit trong kafka-go .....                  | 6  |
| 1.4 Thực hành: 2 Consumer cùng Group .....                      | 7  |
| Chạy thử và quan sát.....                                       | 9  |
| Chương 2: Goroutine, Channel & Context với Kafka .....          | 10 |
| 2.1 Goroutine — Thread siêu nhẹ của Go.....                     | 10 |
| 2.2 Channel — Cách goroutine giao tiếp an toàn.....             | 10 |
| 2.3 context.Context — Người kiểm soát thời gian.....            | 11 |
| 2.4 Pattern thực tế: Consumer Pool với Goroutine + Channel..... | 11 |
| 2.5 Graceful Shutdown — Dừng lại sạch sẽ.....                   | 13 |
| Chương 3: Xử Lý Lỗi Nâng Cao & Retry Pattern.....               | 15 |
| 3.1 Phân loại lỗi Kafka .....                                   | 15 |
| 3.2 Exponential Backoff — Retry thông minh .....                | 15 |
| 3.3 Dead Letter Queue (DLQ) .....                               | 16 |
| 3.4 Idempotent Producer — Gửi không trùng lặp.....              | 18 |
| 3.5 Structured Logging với log/slog .....                       | 18 |
| Chương 4: Serialization JSON & Schema Versioning .....          | 20 |
| 4.1 Tại sao không gửi raw string? .....                         | 20 |
| 4.2 JSON với Struct Tag và Validation .....                     | 20 |
| 4.3 Schema Versioning — Không bao giờ break consumer cũ.....    | 21 |
| 4.4 Giới thiệu Schema Registry (lý thuyết) .....                | 22 |
| Bài Tập Tổng Hợp Cuối Chương .....                              | 24 |
| Đề bài: Mini Order Processing Pipeline .....                    | 24 |
| Gợi ý từng bước (không đưa đáp án) .....                        | 24 |
| Bước 1: Chuẩn bị models .....                                   | 24 |
| Bước 2: Tạo topic với nhiều partition.....                      | 24 |
| Bước 3: Xây dựng Consumer chính .....                           | 24 |
| Bước 4: Xây dựng Consumer thống kê .....                        | 25 |
| Bước 5: Chạy và quan sát .....                                  | 25 |
| Lỗi Thường Gặp & Cách Xử Lý (Tổng Hợp).....                     | 26 |
| Tổng Kết & Bảng Ôn Tập Khái Niệm .....                          | 27 |
| Bảng ôn tập khái niệm .....                                     | 27 |

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Lộ trình Bài 3: Schema Registry, Kafka Streams, Exactly-Once ..... | 27 |
|--------------------------------------------------------------------|----|

# Ôn Tập Nhanh & Folder Structure Mở Rộng

Bài này tiếp nối trực tiếp bài 1. Bạn đã có: module Go, thư viện kafka-go, Producer/Consumer đơn giản, Kafka đang chạy qua docker-compose. Chúng ta không làm lại — chỉ mở rộng.

## Folder structure mở rộng (phần mới in đậm)

Cấu trúc thư mục bài 2

```
kafka-go-demo/
├── docker-compose.yml
├── go.mod / go.sum
├── cmd/
│   ├── producer/
│   │   └── main.go           ← Bài 1 (sẽ mở rộng thêm)
│   ├── consumer/
│   │   └── main.go           ← Bài 1 (sẽ mở rộng thêm)
│   ├── consumer-group/      ← MỚI: chạy 2 consumer cùng group
│   │   └── main.go
│   ├── dlq-consumer/        ← MỚI: consumer đọc Dead Letter Queue
│   │   └── main.go
│   └── internal/
│       └── kafka/
│           ├── producer.go    ← Bài 1 (bổ sung idempotent)
│           ├── consumer.go    ← Bài 1 (bổ sung manual commit)
│           ├── consumer_group.go ← MỚI: consumer group pool
│           └── retry.go        ← MỚI: exponential backoff + DLQ
├── config/config.go
├── models/
│   ├── order.go              ← Bài 1
│   └── event.go               ← MỚI: versioned event wrapper
```

# Chương 1: Consumer Group & Offset Management

Ở bài 1, bạn đã chạy 1 consumer đọc message từ Kafka. Nhưng trong thực tế, một consumer đơn lẻ sẽ trở thành nút thắt cổ chai khi lượng message tăng lên. Consumer Group giải quyết bài toán này.

## 1.1 Consumer Group là gì?

Hãy tưởng tượng quán cà phê giờ cao điểm: 500 order dồn vào, mà chỉ có 1 nhân viên pha chế. Khách chờ dài cả cây số. Giải pháp? Thuê thêm nhân viên — nhưng cần có người điều phối để không ai làm trùng việc của người kia.

Consumer Group chính là cơ chế điều phối đó trong Kafka. Mỗi Consumer trong cùng một Group sẽ được phân công đọc từ một tập Partition riêng biệt — không ai đọc trùng với ai.

Topic "don-hang" có 3 Partition

| Consumer Group "email-svc" |                           |
|----------------------------|---------------------------|
| Partition 0                | → Consumer A (instance 1) |
| Partition 1                | → Consumer B (instance 2) |
| Partition 2                | → Consumer C (instance 3) |

Consumer Group "sms-svc" (đọc lập — đọc lại từ đầu)

|             |              |
|-------------|--------------|
| Partition 0 | → Consumer X |
| Partition 1 | → Consumer Y |
| Partition 2 | → Consumer Z |

→ Cùng 1 topic, 2 group khác nhau đọc HOÀN TOÀN ĐỘC LẬP.  
→ email-svc và sms-svc đều nhận đủ 100% message.

### 💡 Mẹo quan trọng

Số Consumer trong một Group nên = số Partition của topic để tận dụng tối đa.  
Nếu Consumer > Partition: consumer thừa sẽ idle (không làm gì).  
Nếu Consumer < Partition: một consumer sẽ xử lý nhiều partition hơn.

## 1.2 Rebalance — Kafka tự cân bằng khi có biến động

Rebalance là quá trình Kafka phân phối lại Partition cho các Consumer khi có sự thay đổi trong Group. Đây là cơ chế tự động — bạn không cần làm gì, nhưng cần hiểu để xử lý đúng.

TÌNH HUỐNG 1: Thêm Consumer vào Group

|                  |                              |
|------------------|------------------------------|
| Trước rebalance: | Sau rebalance:               |
| P0 → Consumer A  | P0 → Consumer A              |
| P1 → Consumer A  | P1 → Consumer B (B vừa join) |
| P2 → Consumer B  | P2 → Consumer C (C vừa join) |

TÌNH HUỐNG 2: Consumer bị crash

```

Trước:                                Sau rebalance (tự động):
P0 → Consumer A                      P0 → Consumer B (B nhận thêm)
P1 → Consumer B                      P1 → Consumer B
P2 → Consumer C                      P2 → Consumer C

→ Kafka phát hiện A không gửi heartbeat → trigger rebalance → B tiếp quản P0
    
```

### ⚠ Rebalance = hệ thống dừng xử lý tạm thời

Trong thời gian rebalance, tất cả consumer trong group tạm ngừng đọc.  
 Message vẫn an toàn trên Kafka — nhưng latency tăng lên.  
 Tần suất rebalance nhiều = dấu hiệu hệ thống không ổn định.  
 session.timeout.ms (mặc định 10s): nếu consumer không gửi heartbeat trong 10s → bị coi là chết → rebalance.

## 1.3 Commit Offset: Tự động vs Thủ công

Offset commit là việc Consumer báo với Kafka: "Tôi đã xử lý xong message đến vị trí này." Đây là một trong những quyết định quan trọng nhất trong thiết kế Consumer.

| Tiêu chí        | Auto Commit                                         | Manual Commit                                |
|-----------------|-----------------------------------------------------|----------------------------------------------|
| Cách hoạt động  | Kafka tự commit định kỳ (mặc định 1s)               | Bạn gọi CommitMessages() sau khi xử lý xong  |
| Rủi ro          | Commit trước khi xử lý xong → mất message nếu crash | Phức tạp hơn, phải tự gọi đúng chỗ           |
| Xử lý trùng lặp | Có thể xảy ra (at-most-once)                        | Kiểm soát được (at-least-once)               |
| Dùng khi        | Log, metrics — mất 1 vài message không sao          | Đơn hàng, thanh toán — không được bỏ sót     |
| Code            | <code>CommitInterval: 1*time.Second</code>          | <code>reader.CommitMessages(ctx, msg)</code> |

### 1.3.1 Code: Manual Commit trong kafka-go

```

internal/kafka/consumer.go

// File: internal/kafka/consumer.go (bổ sung từ bài 1)

// NewConsumerManualCommit tạo Consumer với commit thủ công.
// Dùng khi xử lý message quan trọng: tài chính, đơn hàng...
func NewConsumerManualCommit(brokers []string, topic, groupID string) *Consumer
{
    reader := kafka.NewReader(kafka.ReaderConfig{
        Brokers: brokers,
        Topic:   topic,
        GroupID: groupID,
        MinBytes: 10e3,
        MaxBytes: 10e6,
        MaxWait: 1 * time.Second,
        // CommitInterval = 0 → tắt auto commit
        // Bạn phải tự gọi CommitMessages() sau khi xử lý thành công
    })
}
    
```

```

        CommitInterval: 0,
    })
    return &Consumer{reader: reader}
}

// ReadAndCommit đọc message VÀ commit offset sau khi xử lý thành công.
// processFn là hàm bạn tự định nghĩa để xử lý message.
func (c *Consumer) ReadAndCommit(
    ctx context.Context,
    processFn func(Message) error,
) error {
    // FetchMessage: chỉ lấy message, CHƯA commit offset
    kafkaMsg, err := c.reader.FetchMessage(ctx)
    if err != nil {
        return fmt.Errorf("fetch thất bại: %w", err)
    }

    msg := Message{
        Key:      string(kafkaMsg.Key),
        Value:    kafkaMsg.Value,
        Partition: kafkaMsg.Partition,
        Offset:   kafkaMsg.Offset,
        Timestamp: kafkaMsg.Time,
        raw:     kafkaMsg, // lưu lại để commit sau
    }

    // Gọi hàm xử lý — nếu lỗi, KHÔNG commit → Kafka sẽ gửi lại
    if err := processFn(msg); err != nil {
        return fmt.Errorf("xử lý lỗi (offset=%d): %w", msg.Offset, err)
    }

    // Chỉ commit SAU KHI xử lý thành công
    if err := c.reader.CommitMessages(ctx, kafkaMsg); err != nil {
        return fmt.Errorf("commit offset thất bại: %w", err)
    }

    return nil
}

```

### ⚠ Thứ tự quan trọng: Xử lý trước, Commit sau

Nếu bạn commit TRƯỚC rồi xử lý → app crash → message bị bỏ qua (mất vĩnh viễn).  
 Nếu bạn commit SAU khi xử lý xong → app crash → Kafka gửi lại message → an toàn hơn.  
 Hệ quả: bạn có thể xử lý trùng message (at-least-once). Cần idempotent ở tầng xử lý.

## 1.4 Thực hành: 2 Consumer cùng Group

Chúng ta sẽ chạy 2 goroutine giả lập 2 Consumer trong cùng một Group và quan sát Kafka tự chia Partition cho từng Consumer.

```

cmd/consumer-group/main.go

// File: cmd/consumer-group/main.go
package main

import (
    "context"
    "encoding/json"
    "fmt"

```

```

    "log"
    "os"
    "os/signal"
    "sync"
    "syscall"
    "time"

    "github.com/segmentio/kafka-go"
    "github.com/hocsinh/kafka-go-demo/config"
    "github.com/hocsinh/kafka-go-demo/models"
)

func main() {
    cfg := config.LoadKafkaConfig()

    ctx, cancel := signal.NotifyContext(
        context.Background(), os.Interrupt, syscall.SIGTERM,
    )
    defer cancel()

    // Chạy 2 consumer trong cùng group "nhom-don-hang"
    // Kafka sẽ tự phân chia partition cho từng consumer
    var wg sync.WaitGroup
    for i := 1; i <= 2; i++ {
        wg.Add(1)
        go func(consumerID int) {
            defer wg.Done()
            runConsumer(ctx, cfg, consumerID)
        }(i)
    }

    wg.Wait()
    log.Println("Tất cả consumer đã dừng.")
}

func runConsumer(ctx context.Context, cfg config.KafkaConfig, id int) {
    reader := kafka.NewReader(kafka.ReaderConfig{
        Brokers:  cfg.Brokers,
        Topic:     cfg.Topic,
        GroupID:   "nhom-don-hang", // cùng GroupID = cùng group
        MinBytes:  10e3,
        MaxBytes:  10e6,
        MaxWait:   500 * time.Millisecond,
    })
    defer reader.Close()

    log.Printf("[Consumer %d] Bắt đầu lắng nghe...", id)

    for {
        msg, err := reader.ReadMessage(ctx)
        if err != nil {
            if ctx.Err() != nil {
                log.Printf("[Consumer %d] Dừng theo lệnh.", id)
                return
            }
            log.Printf("[Consumer %d] Lỗi đọc: %v", id, err)
            continue
        }

        var order models.Order
        if err := json.Unmarshal(msg.Value, &order); err != nil {
            log.Printf("[Consumer %d] JSON lỗi: %v", id, err)
            continue
        }
    }
}

```



```
// In rõ consumer nào xử lý partition nào
fmt.Printf("[Consumer %d] Partition=%d Offset=%d OrderID=%s\n",
    id, msg.Partition, msg.Offset, order.ID)
}
}
```

## Chạy thử và quan sát

```
# Terminal A: Chạy consumer group
go run ./cmd/consumer-group/

# Terminal B: Gửi nhiều message
go run ./cmd/producer/

# Quan sát output Terminal A — bạn sẽ thấy:
# [Consumer 1] Partition=0 Offset=0 OrderID=ORD-001
# [Consumer 2] Partition=1 Offset=0 OrderID=ORD-002
# [Consumer 1] Partition=0 Offset=1 OrderID=ORD-003
# → Consumer 1 luôn xử lý Partition 0
# → Consumer 2 luôn xử lý Partition 1
# → Không có message nào bị xử lý trùng!
```

### Tóm tắt chương

Consumer Group = nhóm consumer chia nhau partition, không ai đọc trùng.  
 Nhiều Group khác nhau đọc cùng topic hoàn toàn độc lập.  
 Rebalance tự xảy ra khi consumer vào/ra khỏi group — hệ thống tạm dừng ngắn.  
 Auto commit: tiện nhưng có thể mất message. Manual commit: an toàn hơn cho hệ thống quan trọng.  
 Số consumer lý tưởng = số partition của topic.

## Chương 2: Goroutine, Channel & Context với Kafka

Đây là chương mà Go thể hiện sức mạnh thực sự của mình. Goroutine và Channel là lý do nhiều team chọn Go để xây dựng hệ thống xử lý message — nhẹ, nhanh, dễ quản lý hơn thread truyền thống rất nhiều.

### 2.1 Goroutine — Thread siêu nhẹ của Go

Thread trong Java hay Python tốn khoảng 1-8MB RAM mỗi cái. Goroutine của Go chỉ bắt đầu với khoảng 2KB và tự mở rộng khi cần. Điều này có nghĩa là trên cùng một máy, bạn có thể chạy hàng chục nghìn goroutine trong khi Java chỉ chạy được vài trăm thread.

```
// So sánh: Thread Java vs Goroutine Go

// Java — mỗi thread tốn ~1MB, tạo 10.000 thread = 10GB RAM
// for (int i = 0; i < 10000; i++) {
//     new Thread() -> processMessage().start(); // nguy hiểm!
// }

// Go — mỗi goroutine ~2KB, tạo 10.000 goroutine = 20MB RAM
for i := 0; i < 10000; i++ {
    go processMessage() // hoàn toàn bình thường trong Go
}

// Cách tạo goroutine đơn giản nhất: thêm từ khóa "go" trước lời gọi hàm
go func() {
    fmt.Println("Tôi chạy song song với hàm main!")
}()
```

### 2.2 Channel — Cách goroutine giao tiếp an toàn

Khi nhiều goroutine cùng chạy, chúng cần cách để truyền dữ liệu cho nhau mà không bị xung đột. Channel chính là "đường ống" để làm việc đó — goroutine A bỏ dữ liệu vào, goroutine B lấy ra, Go đảm bảo an toàn tuyệt đối.

```
// Tạo channel chứa kiểu string, buffer tối đa 10 phần tử
msgChan := make(chan string, 10)

// Goroutine 1: GỬI dữ liệu vào channel
go func() {
    msgChan <- "đơn hàng ORD-001" // gửi vào
    msgChan <- "đơn hàng ORD-002"
    close(msgChan) // đóng khi xong, báo hiệu không còn data
}()

// Goroutine 2 (hoặc main): NHẬN dữ liệu từ channel
for msg := range msgChan { // tự dừng khi channel bị close
    fmt.Println("Xử lý:", msg)
}

// Unbuffered channel (không có buffer): gửi sẽ BLOCK cho đến khi
// có goroutine khác sẵn sàng nhận
syncChan := make(chan int) // buffer = 0
```

### 💡 Buffered vs Unbuffered Channel

make(chan T): unbuffered — gửi block ngay nếu chưa có ai nhận.

make(chan T, N): buffered N phần tử — gửi không block cho đến khi đầy.

Với Kafka consumer, dùng buffered channel để tránh consumer bị block khi worker bận.

## 2.3 context.Context — Người kiểm soát thời gian

Hãy tưởng tượng bạn gọi pizza. Bạn muốn nếu sau 30 phút mà chưa giao thì hủy đơn. Context.Context làm đúng việc đó cho các goroutine — bạn đặt ra một "deadline" hoặc "cancel signal", và tất cả goroutine đang chờ sẽ nhận được thông báo để dừng lại.

```
// Pattern 1: Timeout — tự hủy sau khoảng thời gian
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel() // LUÔN phải defer cancel để giải phóng tài nguyên!

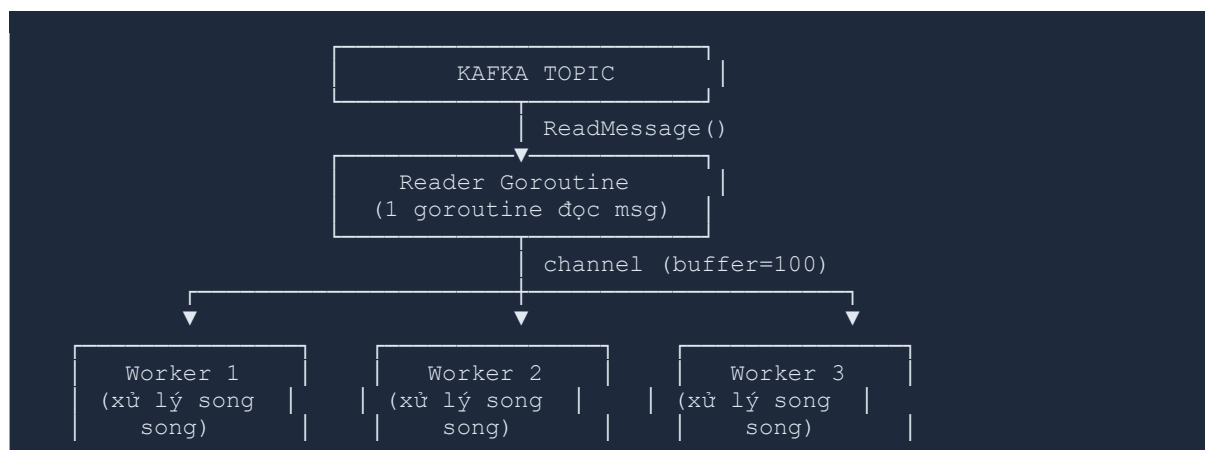
// Pattern 2: Deadline — hủy vào thời điểm cụ thể
deadline := time.Now().Add(10 * time.Minute)
ctx, cancel := context.WithDeadline(context.Background(), deadline)
defer cancel()

// Pattern 3: Cancel thủ công — hủy khi bạn muốn
ctx, cancel := context.WithCancel(context.Background())
go func() {
    // Chạy cho đến khi nhận signal Ctrl+C
    <-shutdownSignal
    cancel() // thông báo tắt cả goroutine dừng lại
}()

// Dùng với Kafka: reader sẽ dừng khi ctx bị cancel
msg, err := reader.ReadMessage(ctx)
if err != nil && ctx.Err() != nil {
    log.Println("Context bị hủy, consumer dừng lại.")
}
```

## 2.4 Pattern thực tế: Consumer Pool với Goroutine + Channel

Đây là pattern phổ biến nhất trong hệ thống thực tế: một goroutine đọc message từ Kafka (reader), nhiều goroutine worker xử lý song song. Giống như băng chuyền nhà máy — một người lấy hàng từ kho, nhiều người cùng đóng gói.



```
internal/kafka/consumer_group.go
```

```
// File: internal/kafka/consumer_group.go
package kafka

import (
    "context"
    "log/slog"
    "sync"

    "github.com/segmentio/kafka-go"
)

// WorkerPool chạy N worker goroutine song song để xử lý message.
type WorkerPool struct {
    reader      *kafka.Reader
    numWorkers  int
    msgChan     chan kafka.Message // channel truyền message từ reader → workers
}

// NewWorkerPool tạo pool với numWorkers goroutine worker.
func NewWorkerPool(reader *kafka.Reader, numWorkers int) *WorkerPool {
    return &WorkerPool{
        reader:      reader,
        numWorkers:  numWorkers,
        // Buffer = numWorkers * 10: tránh reader bị block khi worker bận
        msgChan:     make(chan kafka.Message, numWorkers*10),
    }
}

// Run khởi động toàn bộ pool.
// processFn: hàm xử lý message, được gọi bởi từng worker.
func (p *WorkerPool) Run(ctx context.Context, processFn func(kafka.Message) error) {
    var wg sync.WaitGroup

    // Bước 1: Khởi động N worker goroutine
    for i := 0; i < p.numWorkers; i++ {
        wg.Add(1)
        workerID := i + 1
        go func() {
            defer wg.Done()
            p.runWorker(ctx, workerID, processFn)
        }()
    }

    // Bước 2: Goroutine reader đọc từ Kafka → bỏ vào channel
    go func() {
        defer close(p.msgChan) // đóng channel khi reader xong
        for {
            msg, err := p.reader.FetchMessage(ctx)
            if err != nil {
                if ctx.Err() != nil {
                    return // context bị hủy, thoát gracefully
                }
                slog.Error("fetch lỗi", "err", err)
                continue
            }
            p.msgChan <- msg // gửi vào channel cho worker xử lý
        }
    }()
}
```

```

    wg.Wait() // chờ tất cả worker xong
}

// runWorker: goroutine worker đọc từ channel và xử lý
func (p *WorkerPool) runWorker(ctx context.Context, id int, fn
func(kafka.Message) error) {
    slog.Info("Worker khởi động", "worker_id", id)

    for msg := range p.msgChan { // tự dừng khi msgChan bị close
        if err := fn(msg); err != nil {
            slog.Error("Xử lý lỗi",
                "worker_id", id,
                "offset", msg.Offset,
                "err", err,
            )
            continue // không commit, Kafka gửi lại
        }
        // Commit sau khi xử lý thành công
        if err := p.reader.CommitMessages(ctx, msg); err != nil {
            slog.Error("Commit lỗi", "worker_id", id, "err", err)
        }
    }

    slog.Info("Worker dừng", "worker_id", id)
}

```

## 2.5 Graceful Shutdown — Dừng lại sạch sẽ

Graceful shutdown là kỹ năng bắt buộc trong production. Khi server nhận lệnh dừng (Ctrl+C, Kubernetes rolling update...), chương trình phải: 1) Ngừng nhận message mới. 2) Xử lý xong các message đang đang dở. 3) Commit offset. 4) Đóng kết nối.

```

cmd/consumer-group/main.go

// File: cmd/consumer-group/main.go (phiên bản graceful shutdown)
package main

import (
    "context"
    "log/slog"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/segmentio/kafka-go"
    "github.com/hocsinh/kafka-go-demo/config"
    kafkaclient "github.com/hocsinh/kafka-go-demo/internal/kafka"
)

func main() {
    cfg := config.LoadKafkaConfig()

    // signal.NotifyContext: khi Ctrl+C hoặc SIGTERM → ctx bị cancel
    ctx, stop := signal.NotifyContext(
        context.Background(),
        os.Interrupt, // Ctrl+C
        syscall.SIGTERM, // kill command / Kubernetes
    )
    defer stop()

```

```

reader := kafka.NewReader(kafka.ReaderConfig{
    Brokers:      cfg.Brokers,
    Topic:        cfg.Topic,
    GroupID:      cfg.GroupID,
    CommitInterval: 0, // manual commit
    MaxWait:      500 * time.Millisecond,
})

pool := kafkaclient.NewWorkerPool(reader, 3) // 3 worker

slog.Info("Consumer pool khởi động", "workers", 3, "topic", cfg.Topic)

// Run block cho đến khi ctx bị cancel (Ctrl+C)
pool.Run(ctx, func(msg kafka.Message) error {
    slog.Info("Xử lý message",
        "partition", msg.Partition,
        "offset", msg.Offset,
        "key", string(msg.Key),
    )
    // ... logic xử lý thực tế ở đây ...
    return nil
})

slog.Info("Graceful shutdown hoàn tất.")
}

```

### Tóm tắt chương

Goroutine nhẹ hơn thread Java ~500 lần — có thể chạy hàng chục nghìn cái.

Channel là đường ống an toàn để goroutine giao tiếp — không cần mutex.

context.Context là bắt buộc — dùng để timeout, cancel, truyền signal dừng.

Pattern Worker Pool: 1 reader goroutine + N worker goroutine qua buffered channel.

Graceful shutdown: nhận SIGTERM → cancel context → worker xử lý xong → đóng kết nối.

## Chương 3: Xử Lý Lỗi Nâng Cao & Retry Pattern

Trong hệ thống thực tế, lỗi là điều tất yếu. Database chậm, API bên thứ ba timeout, network bị ngắt... Hệ thống tốt không phải là hệ thống không có lỗi, mà là hệ thống biết cách xử lý lỗi một cách thông minh.

### 3.1 Phân loại lỗi Kafka

Không phải lỗi nào cũng xử lý giống nhau. Retry một lỗi "không thể sửa" là lãng phí tài nguyên và sẽ làm nghẽn toàn bộ pipeline.

| Loại lỗi              | Ví dụ                                  | Xử lý                                         |
|-----------------------|----------------------------------------|-----------------------------------------------|
| Transient (tạm thời)  | Network timeout, DB quá tải tạm thời   | Retry với backoff — thường tự hết sau vài lần |
| Permanent (vĩnh viễn) | JSON sai format, business rule vi phạm | Không retry — gửi vào Dead Letter Queue       |
| Kafka infrastructure  | Broker không phản hồi, leader election | Retry nhưng với interval dài hơn              |

### 3.2 Exponential Backoff — Retry thông minh

Retry liên tục (ngay lập tức) là anti-pattern: nếu DB đang quá tải, bạn retry 100 lần/giây sẽ làm nó càng chậm hơn. Exponential backoff giải quyết bằng cách tăng dần thời gian chờ giữa mỗi lần retry.

```
Công thức: wait = baseDelay * 2^attempt + jitter

Attempt 1: wait = 1s * 2^0 + random(0-1s) ≈ 1-2s
Attempt 2: wait = 1s * 2^1 + random(0-1s) ≈ 2-3s
Attempt 3: wait = 1s * 2^2 + random(0-1s) ≈ 4-5s
Attempt 4: wait = 1s * 2^3 + random(0-1s) ≈ 8-9s
Attempt 5: wait = min(1s * 2^4, maxDelay=30s) ≈ 16-17s

→ "jitter" (ngẫu nhiên) để tránh nhiều consumer retry cùng lúc (thundering herd)
```

```
internal/kafka/retry.go

// File: internal/kafka/retry.go
package kafka

import (
    "context"
    "errors"
    "fmt"
    "log/slog"
    "math"
    "math/rand"
    "time"
)

// RetryConfig cấu hình cho cơ chế retry
```

```

type RetryConfig struct {
    MaxAttempts int          // số lần thử tối đa (kể cả lần đầu)
    BaseDelay   time.Duration // thời gian chờ cơ sở
    MaxDelay    time.Duration // giới hạn trên của thời gian chờ
}

// DefaultRetryConfig là cấu hình hợp lý cho hầu hết trường hợp
var DefaultRetryConfig = RetryConfig{
    MaxAttempts: 5,
    BaseDelay:   1 * time.Second,
    MaxDelay:    30 * time.Second,
}

// WithRetry thực thi fn với exponential backoff retry.
// Trả về lỗi cuối cùng nếu hết số lần thử.
func WithRetry(ctx context.Context, cfg RetryConfig, fn func() error) error {
    var lastErr error

    for attempt := 0; attempt < cfg.MaxAttempts; attempt++ {
        // Gọi hàm cần thực thi
        if err := fn(); err == nil {
            return nil // thành công, thoát ngay
        } else {
            lastErr = err
            slog.Warn("Thực thi thất bại, sẽ retry",
                "attempt", attempt+1,
                "max",    cfg.MaxAttempts,
                "err",    err,
            )
        }

        // Đây là lần thử cuối, không cần chờ
        if attempt == cfg.MaxAttempts-1 {
            break
        }

        // Tính thời gian chờ: baseDelay * 2^attempt
        delay := float64(cfg.BaseDelay) * math.Pow(2, float64(attempt))
        // Giới hạn delay không vượt quá MaxDelay
        if delay > float64(cfg.MaxDelay) {
            delay = float64(cfg.MaxDelay)
        }
        // Thêm jitter: ±25% để tránh thundering herd
        jitter := delay * 0.25 * (rand.Float64()*2 - 1)
        waitDuration := time.Duration(delay + jitter)

        slog.Info("Chờ trước khi retry", "wait", waitDuration)

        // Chờ — nhưng thoát ngay nếu context bị cancel
        select {
        case <-time.After(waitDuration):
            // tiếp tục retry
        case <-ctx.Done():
            return fmt.Errorf("context bị hủy trong khi retry: %w", ctx.Err())
        }
    }

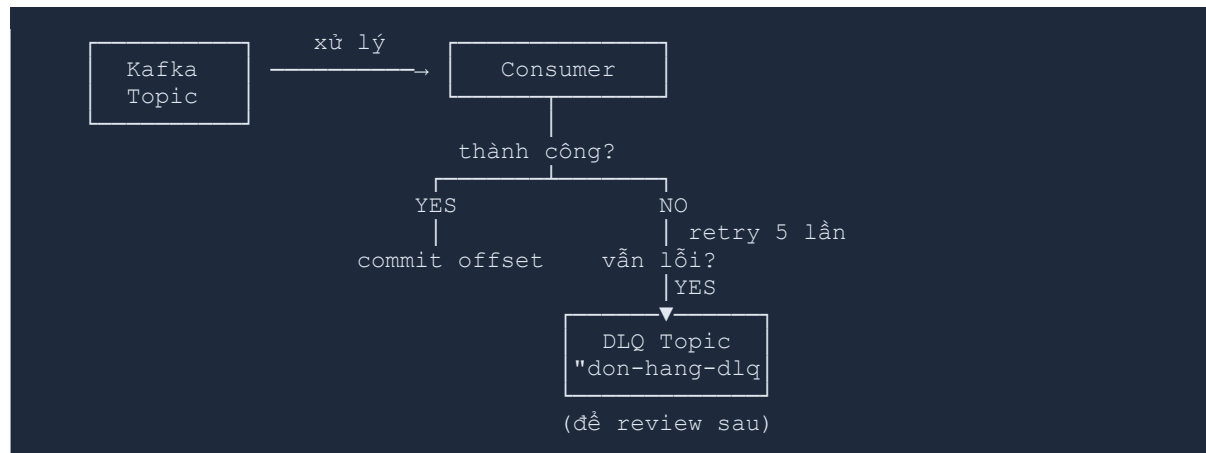
    return fmt.Errorf("thất bại sau %d lần thử: %w", cfg.MaxAttempts, lastErr)
}

```

### 3.3 Dead Letter Queue (DLQ)



DLQ là một topic Kafka riêng dùng để chứa các message không thể xử lý sau nhiều lần retry. Thay vì mất message hoặc làm nghẽn pipeline, bạn "bãi đổ" chúng ở DLQ để xem xét sau.



`internal/kafka/retry.go`

```
// File: internal/kafka/retry.go (tiếp theo)

// DLQSender gửi message lỗi vào Dead Letter Queue topic
type DLQSender struct {
    writer *kafka.Writer
}

func NewDLQSender(brokers []string, dlqTopic string) *DLQSender {
    return &DLQSender{
        writer: &kafka.Writer{
            Addr:      kafka.TCP(brokers...),
            Topic:     dlqTopic,
            AllowAutoTopicCreation: true,
        },
    }
}

// Send gửi message gốc vào DLQ kèm thông tin lỗi.
// originalMsg: message gốc không xử lý được
// reason: lý do thất bại (dùng để debug sau)
func (d *DLQSender) Send(ctx context.Context, originalMsg kafka.Message, reason error) error {
    // Đóng gói message gốc + thông tin lỗi vào DLQ
    dlqPayload := struct {
        OriginalKey   string `json:"original_key"`
        OriginalValue string `json:"original_value"`
        ErrorReason   string `json:"error_reason"`
        FailedAt     string `json:"failed_at"`
    }{
        OriginalKey:   string(originalMsg.Key),
        OriginalValue: string(originalMsg.Value),
        ErrorReason:   reason.Error(),
        FailedAt:     time.Now().Format(time.RFC3339),
    }

    payloadBytes, _ := json.Marshal(dlqPayload)

    return d.writer.WriteMessages(ctx, kafka.Message{
        Key:   originalMsg.Key, // giữ nguyên key để trace
        Value: payloadBytes,
        // Thêm header để dễ filter trong Redpanda Console
    })
}
```

```

        Headers: []kafka.Header{
            {Key: "x-error-reason", Value: []byte(reason.Error())},
            {Key: "x-original-topic", Value: []byte(originalMsg.Topic)},
        },
    ))
}

func (d *DLQSender) Close() error { return d.writer.Close() }

```

## 3.4 Idempotent Producer — Gửi không trùng lặp

Khi network gặp sự cố, Producer có thể gửi lại message mà không biết lần đầu đã thành công chưa. Idempotent producer đảm bảo dù gửi bao nhiêu lần, Kafka chỉ lưu đúng một bản.

```

internal/kafka/producer.go

// File: internal/kafka/producer.go (bổ sung)

// NewIdempotentProducer tạo producer đảm bảo không gửi trùng.
// Kafka sẽ tự deduplicate nếu cùng sequence number + producer ID.
func NewIdempotentProducer(brokers []string, topic string) *Producer {
    writer := &kafka.Writer{
        Addr:      kafka.TCP(brokers...),
        Topic:     topic,
        Balancer:  &kafka.Hash{}, // cùng key → cùng partition (cần cho idempotent)

        // RequireAll = chờ tất cả replica xác nhận
        // Cần thiết để idempotent hoạt động đúng
        RequiredAcks: kafka.RequireAll,

        // MaxAttempts: số lần kafka-go tự retry nội bộ
        MaxAttempts: 3,

        WriteTimeout: 10 * time.Second,
        ReadTimeout:  10 * time.Second,
        AllowAutoTopicCreation: true,
    }
    return &Producer{writer: writer}
}

```

## 3.5 Structured Logging với log/slog

Go 1.21 giới thiệu log/slog — structured logging chính thức. Với Kafka, log có cấu trúc giúp bạn dễ dàng filter và tìm kiếm: "tìm tất cả lỗi liên quan đến partition 2 trong 1 giờ qua".

```

// Thay vì log.Printf("Lỗi xử lý message offset %d: %v", offset, err)
// Dùng slog với key-value pairs:

slog.Error("Xử lý message thất bại",
    "topic",      msg.Topic,
    "partition",  msg.Partition,
    "offset",     msg.Offset,
    "key",        string(msg.Key),
    "err",        err,
)

```

```
// Output (JSON format, dễ đưa vào Elasticsearch/Loki):
// {
//   "time": "2025-04-01T10:05:23Z",
//   "level": "ERROR",
//   "msg": "Xử lý message thất bại",
//   "topic": "don-hang",
//   "partition": 2,
//   "offset": 1042,
//   "key": "ORD-999",
//   "err": "database: connection refused"
// }

// Cấu hình JSON output trong main():
slog.SetDefault(slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelInfo,
})))
```

### Tóm tắt chương

Phân biệt transient vs permanent error để chọn chiến lược xử lý phù hợp.  
 Exponential backoff: tăng dần thời gian chờ giữa retry + jitter ngẫu nhiên.  
 Dead Letter Queue: nơi "bãi đỗ" message không xử lý được, không làm mất data.  
 Idempotent producer: RequireAll acks + Hash balancer để tránh gửi trùng.  
 log/slog: structured logging bắt buộc trong production, dễ tìm kiếm và filter.

## Chương 4: Serialization JSON & Schema Versioning

Kafka lưu message dưới dạng byte array — nó không quan tâm bên trong là gì. Nhưng Producer và Consumer phải đồng thuận về định dạng. Chương này đi sâu vào cách làm điều đó đúng cách.

### 4.1 Tại sao không gửi raw string?

|                     | Raw String                         | JSON Struct                             |
|---------------------|------------------------------------|-----------------------------------------|
| Validate dữ liệu    | Không — bất kỳ chuỗi nào cũng qua  | Compile-time check qua struct tag       |
| Refactor field name | Phải tìm/thay thủ công, dễ sót     | Rename struct field, build báo lỗi ngay |
| Thêm/bỏ field       | Consumer cũ bị crash nếu parse sai | json:"omitempty" xử lý tự động          |
| Đọc log / debug     | Có thể, nhưng dễ nhầm              | JSON đọc được ngay, tool hỗ trợ tốt     |
| Hiệu năng           | Nhanh hơn một chút                 | Đủ nhanh cho 99% use case               |

### 4.2 JSON với Struct Tag và Validation

models/event.go

```
// File: models/event.go - Event wrapper có version
package models

import "time"

// EventEnvelope bọc mọi message gửi qua Kafka.
// Đây là pattern quan trọng: tách "metadata" khỏi "payload".
type EventEnvelope struct {
    // Version cho phép Consumer xử lý đúng theo schema cũ/mới
    Version int `json:"version"`

    // EventType giúp Consumer biết cần deserialize Payload thành struct nào
    EventType string `json:"event_type"`

    // CorrelationID: dùng để trace 1 request xuyên suốt nhiều service
    CorrelationID string `json:"correlation_id"`

    // Payload là nội dung thực tế - dùng json.RawMessage để defer parse
    Payload json.RawMessage `json:"payload"`

    // PublishedAt ghi lại thời điểm Producer gửi
    PublishedAt time.Time `json:"published_at"`
}

// OrderEventV1 là schema version 1
type OrderEventV1 struct {
    OrderID string `json:"order_id"`
    CustomerID string `json:"customer_id"`
    Amount float64 `json:"amount"`
    Status string `json:"status"`
}
```

```

}

// OrderEventV2 là schema version 2 - thêm field ShippingAddress
// Field mới dùng omitempty để không break consumer đọc V1
type OrderEventV2 struct {
    OrderID      string `json:"order_id"`
    CustomerID   string `json:"customer_id"`
    Amount       float64 `json:"amount"`
    Status       string `json:"status"`
    // Field mới - omitempty: nếu rỗng thì không include trong JSON
    ShippingAddress string `json:"shipping_address,omitempty"`
    Priority      int    `json:"priority,omitempty"`
}

```

### 4.3 Schema Versioning — Không bao giờ break consumer cũ

Đây là một trong những vấn đề khó nhất trong hệ thống microservices. Producer và Consumer được deploy độc lập — bạn không thể đảm bảo tất cả consumer đã được update trước khi producer gửi schema mới.

QUY TẮC VÀNG khi thay đổi schema:

- ☒ **AN TOÀN:**
  - Thêm field MỚI (optional, có default value)
  - Đổi tên field nhưng giữ cả tên cũ (thêm alias)
  - Mở rộng kiểu dữ liệu (int → int64)
- ☒ **NGUY HIỂM (cần versioning):**
  - Xóa field mà consumer cũ đang đọc
  - Đổi kiểu dữ liệu không tương thích (string → int)
  - Đổi ý nghĩa của field cũ

 cmd/producer/main.go + cmd/consumer/main.go

```

// File: cmd/producer/main.go - Producer gửi event có version

func sendOrderEvent(ctx context.Context, p *kafkaclient.Producer, order
models.Order) error {
    // Bước 1: Tạo payload V2
    payload := models.OrderEventV2{
        OrderID:      order.ID,
        CustomerID:   order.CustomerID,
        Amount:       order.TotalAmount,
        Status:       order.Status,
        ShippingAddress: "123 Nguyễn Huệ, Q1, TP.HCM",
        Priority:      1,
    }

    payloadBytes, err := json.Marshal(payload)
    if err != nil {
        return err
    }

    // Bước 2: Bọc vào EventEnvelope với version = 2
    envelope := models.EventEnvelope{
        Version:      2,
        EventType:    "order.created",
    }

```

```

        CorrelationID: uuid.New().String(),
        Payload:        payloadBytes,
        PublishedAt:    time.Now(),
    }

    return p.SendMessage(ctx, order.ID, envelope)
}

// File: cmd/consumer/main.go — Consumer xử lý nhiều version

func processEvent(msg kafkaclient.Message) error {
    // Bước 1: Parse envelope (chung cho mọi version)
    var envelope models.EventEnvelope
    if err := json.Unmarshal(msg.Value, &envelope); err != nil {
        return fmt.Errorf("parse envelope lỗi: %w", err)
    }

    // Bước 2: Route theo version
    switch envelope.Version {
    case 1:
        var order models.OrderEventV1
        if err := json.Unmarshal(envelope.Payload, &order); err != nil {
            return err
        }
        return handleV1(order)

    case 2:
        var order models.OrderEventV2
        if err := json.Unmarshal(envelope.Payload, &order); err != nil {
            return err
        }
        return handleV2(order)

    default:
        // Version không biết → không crash, ghi log và skip
        slog.Warn("Version không hỗ trợ, bỏ qua",
            "version", envelope.Version,
            "event_type", envelope.EventType,
        )
        return nil
    }
}

```

## 4.4 Giới thiệu Schema Registry (lý thuyết)

Cách làm trên (versioning thủ công trong code) hoạt động tốt cho hệ thống vừa. Nhưng khi có hàng chục service, hàng trăm event type, bạn cần công cụ tập trung hơn: Schema Registry.

| Không có Schema Registry:                                                                                  | Có Schema Registry:                                                                                                  |
|------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Producer → Kafka → Consumer<br>Schema nằm trong code mỗi service (registry)<br>Dễ lệch nhau, khó kiểm soát | Producer → [Registry] → Kafka<br>Consumer ————— (lấy schema từ registry)<br>Schema được version & validate tập trung |

### Schema Registry trong bài 3

Bài 3 sẽ đi thực hành Schema Registry với Confluent (tích hợp sẵn trong Redpanda).  
 Bạn sẽ dùng Avro hoặc Protobuf thay vì JSON thuần — nhỏ hơn, nhanh hơn, có schema validation.

Bài này chuẩn bị tư duy: hiểu tại sao cần Schema Registry trước khi cài tool.

### Tóm tắt chương

Luôn dùng struct + JSON thay vì raw string — an toàn hơn, dễ refactor hơn.

EventEnvelope pattern: tách metadata (version, type) khỏi payload thực tế.

Quy tắc vàng: chỉ thêm field optional, không xóa/đổi field cũ đột ngột.

Schema versioning cho phép producer/consumer deploy độc lập mà không break nhau.

Schema Registry (bài 3): giải pháp tập trung cho hệ thống nhiều service phức tạp.

# Bài Tập Tổng Hợp Cuối Chương

## 🎯 Mục tiêu bài tập

Áp dụng kết hợp 4 chương: Consumer Group, Goroutine, Retry + DLQ, JSON Schema.  
Xây dựng mini pipeline hoàn chỉnh có thể chạy được.  
Thực hành quan sát hệ thống qua Redpanda Console.

## Đề bài: Mini Order Processing Pipeline

Xây dựng một pipeline xử lý đơn hàng với các yêu cầu:

1. Producer gửi 20 đơn hàng lên topic "don-hang" dưới dạng EventEnvelope JSON (V2 schema). Một số đơn hàng được "giả lập lỗi" (ví dụ: amount âm).
2. 2 Consumer Group đọc song song từ cùng topic "don-hang":
  - 2.1. Group "xu-ly-don-hang": 3 worker goroutine, xử lý chính (lưu log, giả lập cập nhật DB).
  - 2.2. Group "thong-ke": 1 worker, chỉ đếm và in thống kê.
3. Đơn hàng lỗi (amount âm hoặc customerID rỗng) phải được retry 3 lần với exponential backoff. Nếu vẫn lỗi → gửi vào DLQ topic "don-hang-dlq".
4. Hệ thống phải graceful shutdown khi nhấn Ctrl+C: xử lý xong message đang dang dở, commit offset, đóng kết nối.

## Gợi ý từng bước (không đưa đáp án)

### Bước 1: Chuẩn bị models

- Tạo file models/event.go với EventEnvelope và OrderEventV2 (đã có ở Chương 4).
- Thêm hàm Validate() vào OrderEventV2 — trả về lỗi nếu amount < 0 hoặc customerID rỗng.

### Bước 2: Tạo topic với nhiều partition

```
# Tạo topic với 3 partition (để 3 worker có thể chia nhau)
docker exec my-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic don-hang \
  --partitions 3 --replication-factor 1

docker exec my-kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic don-hang-dlq \
  --partitions 1 --replication-factor 1
```

### Bước 3: Xây dựng Consumer chính

- Dùng WorkerPool (Chương 2) với 3 goroutine.
- Trong processFn: parse EventEnvelope → validate → nếu lỗi dùng WithRetry (Chương 3) → nếu hết retry thì DLQSender.Send().



- Nhớ setup slog JSON handler trong main().

#### Bước 4: Xây dựng Consumer thống kê

- Group ID khác → đọc lại từ đầu hoàn toàn độc lập.
- Dùng sync.Mutex để cập nhật counter an toàn từ nhiều goroutine.
- In thống kê mỗi 5 giây: tổng nhận, tổng thành công, tổng lỗi.

#### Bước 5: Chạy và quan sát

```
# Terminal 1: Consumer chính
go run ./cmd/consumer-group/

# Terminal 2: Consumer thống kê
KAFKA_GROUP_ID=thong-ke go run ./cmd/consumer-group/

# Terminal 3: Producer
go run ./cmd/producer/

# Redpanda Console: http://localhost:9091
# → Xem topic don-hang và don-hang-dlq
# → Xem consumer groups và lag của từng group
```

#### ⚠ Những điểm dễ sai trong bài tập

Quên close DLQSender → message DLQ không được flush ra Kafka.  
 Không dùng json.RawMessage trong EventEnvelope → parse 2 lần bị lỗi.  
 Counter thống kê không dùng mutex → race condition khi nhiều goroutine cùng update.  
 Không test graceful shutdown: nhấn Ctrl+C khi đang xử lý và xem log.

## Lỗi Thường Gặp & Cách Xử Lý (Tổng Hợp)

| Lỗi / Triệu chứng                              | Nguyên nhân                                       | Giải pháp                                        |
|------------------------------------------------|---------------------------------------------------|--------------------------------------------------|
| Consumer không nhận thêm message sau khi scale | Số consumer > số partition → consumer thừa idle   | Tạo topic với partition = số consumer tối đa cần |
| Message bị xử lý 2 lần sau restart             | Auto commit — commit trước khi xử lý xong         | Dùng manual commit (CommitInterval: 0)           |
| Goroutine leak — RAM tăng dần                  | Goroutine tạo ra nhưng không có cơ chế dừng       | Luôn dùng context để cancel goroutine            |
| Channel deadlock — chương trình treo           | Gửi vào full channel hoặc không ai nhận           | Dùng buffered channel + select với ctx.Done()    |
| DLQ không nhận được message                    | Quên gọi DLQSender.Close() → buffer chưa flush    | defer dlq.Close() ngay sau khi tạo               |
| json: cannot unmarshal X into Y                | Schema không khớp giữa producer và consumer       | Dùng EventEnvelope + version routing             |
| Rebalance liên tục (storm)                     | Consumer không gửi heartbeat kịp (xử lý quá chậm) | Tăng max.poll.interval hoặc giảm batch size      |
| slog không in JSON                             | Chưa gọi slog.SetDefault() với JSONHandler        | Thêm vào đầu main(): slog.SetDefault(...)        |

# Tổng Kết & Bảng Ôn Tập Khái Niệm

## Bảng ôn tập khái niệm

| Khái niệm           | Một câu mô tả                                         | Dùng khi                                         |
|---------------------|-------------------------------------------------------|--------------------------------------------------|
| Consumer Group      | Nhiều consumer chia nhau partition, không đọc trùng   | Scale xử lý, HA cho consumer                     |
| Rebalance           | Kafka tự phân lại partition khi consumer vào/ra group | Tự động, cần biết để debug latency spike         |
| Manual Commit       | Chỉ commit offset sau khi xử lý thành công            | Hệ thống quan trọng: đơn hàng, thanh toán        |
| Goroutine           | Luồng siêu nhẹ ~2KB, chạy bằng từ khóa go             | Mọi tác vụ bất đồng bộ trong Go                  |
| Channel             | Đường ống an toàn để goroutine giao tiếp              | Truyền dữ liệu giữa reader và workers            |
| context.Context     | Cơ chế timeout, cancel xuyên suốt call stack          | Bắt buộc cho mọi I/O: Kafka, DB, HTTP            |
| Worker Pool         | 1 reader goroutine + N worker qua buffered channel    | Xử lý message song song với kiểm soát            |
| Graceful Shutdown   | Nhận SIGTERM → dừng nhận mới → xử lý xong → đóng      | Production bắt buộc                              |
| Exponential Backoff | Tăng dần thời gian chờ giữa retry + jitter            | Retry transient error                            |
| Dead Letter Queue   | Topic chứa message không xử lý được sau nhiều retry   | Không muốn mất data nhưng cũng không block       |
| EventEnvelope       | Wrapper chứa version + type + payload                 | Hệ thống có nhiều loại event hoặc cần versioning |
| Schema Versioning   | Thêm field optional, không xóa field cũ               | Khi cần thay đổi schema mà không break consumer  |

## Lộ trình Bài 3: Schema Registry, Kafka Streams, Exactly-Once

Sau khi nắm chắc bài này, bài 3 sẽ đưa bạn vào lãnh thổ hệ thống phân tán thực sự:

- **Schema Registry thực hành:** Cài Confluent Schema Registry (có sẵn trong Redpanda), dùng Protobuf để định nghĩa schema, tự động validate khi Producer gửi sai.
- **Kafka Streams / Stream Processing:** Đọc từ topic A, biến đổi dữ liệu (filter, aggregate, join), ghi vào topic B — trong một pipeline Go dùng goroutine và channel.
- **Exactly-Once Semantics (EOS):** Đảm bảo mỗi message được xử lý đúng một lần — không mất, không trùng. Cần transaction producer + idempotent consumer.
- **Monitoring thực tế:** Xuất Kafka metrics ra Prometheus, vẽ dashboard Grafana để theo dõi consumer lag, throughput, error rate.

— Hết Bài 2 —

## Code nhiều, đọc log kỹ, và đừng sợ lỗi! 🚀